Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2025

SISTEMA DE AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS

202302232 – Pablo Daniel Velásquez Hernández

Resumen

Este ensayo describe el desarrollo de un sistema para la autorización de Documentos Tributarios Electrónicos (DTEs), diseñado para la Superintendencia de Administración Tributaria (SAT) como parte del Proyecto 3 de Introducción a la Programación y Computación 2. Implementado en Python con Programación Orientada a Objetos (POO), utiliza Flask para una API HTTP, Django para una interfaz web, y archivos XML para almacenamiento, reemplazando bases de datos tradicionales. Se empleó un Tipo de Dato Abstracto (TDA) personalizado, ListaSolicitudes, para gestionar solicitudes DTE, validando NITs, IVA, y totales mediante expresiones regulares y algoritmos específicos. El sistema genera números de autorización únicos y reportes gráficos con Chart.js. Este enfoque es vigente en el contexto guatemalteco, donde la digitalización tributaria es clave. Se concluye que la integración de frameworks modernos con POO optimiza la escalabilidad y mantenimiento, aunque presenta retos en la curva de aprendizaje para principiantes.

Palabras clave

Programación Orientada a Objetos, Flask, Django, XML, Documentos Tributarios Electrónicos.

Abstract

This essay outlines the development of a system for authorizing Electronic Tax Documents (DTEs), designed for the Superintendency of Tax Administration (SAT) as part of the IPC2 Project 3. Implemented in Python using Object-Oriented Programming (OOP), it leverages Flask for an HTTP API, Django for a web interface, and XML files for storage, replacing traditional databases. A custom Abstract Data Type (ADT), ListaSolicitudes, manages DTE requests, validating NITs, VAT, and totals using regular expressions and specific algorithms. The system generates unique authorization numbers and graphical reports with Chart.js. This approach is relevant in Guatemala's context, where tax digitization is critical. It is concluded that integrating modern frameworks with OOP enhances scalability and maintainability, though it poses challenges in the learning curve for beginners.

Keywords

Object-Oriented Programming, Flask, Django, XML, Electronic Tax Documents.

Introducción

La digitalización de procesos tributarios es una prioridad en Guatemala, donde la Superintendencia de Administración Tributaria (SAT) impulsa sistemas electrónicos para agilizar la autorización de Documentos Tributarios Electrónicos (DTEs). Este ensayo presenta el desarrollo de un sistema que procesa solicitudes DTE, genera números de autorización únicos, y produce reportes gráficos, implementado como parte del Proyecto 3 de Introducción a la Programación y Computación 2. Utilizando Python, Programación Orientada a Objetos (POO), y frameworks como Flask y Django, el sistema almacena datos en archivos XML, reemplazando bases de datos tradicionales. Su relevancia radica en su alineación con la modernización tributaria y su enfoque educativo en POO y Tipos de Datos Abstractos (TDA). Este trabajo busca responder: ¿Cómo integrar POO, Flask, Django, y XML para crear un sistema eficiente? Se aportan reflexiones técnicas y propuestas para optimizar sistemas similares.

Desarrollo del tema

a. Diseño Orientado a Objetos y Estructura del Sistema

El sistema se diseñó utilizando Programación Orientada a Objetos para garantizar modularidad y reutilización. Se definieron clases clave: SolicitudDTE (almacena datos de una solicitud: fecha, referencia, NITs, valor, IVA, total), ListaSolicitudes (TDA para gestionar solicitudes con métodos para agregar, validar, y autorizar), ValidadorNIT (implementa el algoritmo de validación de NIT), y Autorizacion (genera códigos en formato yyyymmdd########). Estas clases encapsulan la lógica del sistema, facilitando su mantenimiento. La arquitectura sigue un modelo cliente-servidor: el backend (Flask) procesa solicitudes vía API HTTP, mientras el frontend (Django, patrón MVT) ofrece una interfaz para cargar XML, consultar datos, y visualizar gráficas con Chart.js. Los datos se almacenan en autorizaciones.xml, cumpliendo con la estructura especificada por la SAT (ver Figura 1 en apéndice).

b. Implementación del Backend con Flask

El backend, desarrollado con Flask, implementa tres endpoints principales:

- /procesar (POST): Recibe un archivo XML, extrae datos con expresiones regulares, valida NITs, IVA (valor * 0.12), total (valor + IVA), y referencias únicas. Almacena resultados en ListaSolicitudes.
- /consultar (GET): Devuelve el contenido de autorizaciones.xml.
- /reset (POST): Borra autorizaciones.xml y reinicia el correlativo diario.

Este algoritmo asegura la integridad de los NITs. La librería xml.etree.ElementTree gestiona la lectura/escritura de XML, ignorando solicitudes con errores de formato, como exige el enunciado.

c. Frontend con Django y Visualización de Datos

El frontend, implementado en Django, utiliza el patrón Modelo-Vista-Template (MVT) para ofrecer una interfaz amigable. Incluye:

- Carga de archivos: Un formulario para subir XML, enviado al endpoint /procesar vía requests.
- Consulta de datos: Muestra autorizaciones.xml en un recuadro de texto.
- Gráficas: Usa Chart.js para generar:
  - Barras: IVA emitido/recibido por NIT en una fecha.
  - Líneas: Valores totales o sin IVA por rango de fechas.
- Reportes PDF: Generados con reportlab, exportando consultas y gráficas.
- Botones: Enviar (procesa XML), Reset (llama a /reset), y Ayuda (información del estudiante y documentación).

Bootstrap asegura un diseño limpio, mientras las plantillas Django simplifican la integración con el backend. La Tabla I resume las funcionalidades del frontend.

Tabla I. *Funcionalidades del Frontend*

CATEGORÍA	CATEGORÍA
Carga de Archivos	Subir XML con solicitudes DTE
Consulta de Datos	Mostrar autorizaciones.xml en recuadro
Gráficas	Visualizar IVA y valores con Chart.js
Reportes PDF	Exportar consultas/gráficas en PDF
Botones	Enviar, Reset, Ayuda

Fuente: elaboración propia, Pablo Velásquez 2025.

d. Desafíos y Soluciones

El desarrollo enfrentó varios retos:

1. **Curva de aprendizaje**: La integración de Flask y Django fue compleja para principiantes. Se resolvió consultando documentación oficial y foros de UEDI.
2. **Procesamiento de XML**: Extraer datos con expresiones regulares requirió pruebas exhaustivas para manejar formatos inconsistentes. La librería xml.etree.ElementTree simplificó la escritura de autorizaciones.xml.
3. **Generación de gráficas**: Chart.js demandó procesar datos dinámicamente en JavaScript. Se implementó una capa intermedia en Django para formatear datos antes de enviarlos al frontend.
4. **Gestión del tiempo**: Los 17 días disponibles exigieron una planificación rigurosa. El plan de trabajo en 4 fases (configuración, backend, frontend, documentación) aseguró un avance incremental.

Estos desafíos destacan la importancia de combinar herramientas modernas con una metodología estructurada. La experiencia refuerza la relevancia de POO para sistemas escalables y la necesidad de mejorar la formación en frameworks web.

Conclusiones

d. Desafíos y Soluciones

El desarrollo enfrentó varios retos:

1. **Curva de aprendizaje**: La integración de Flask y Django fue compleja para principiantes. Se resolvió consultando documentación oficial y foros de UEDI.
2. **Procesamiento de XML**: Extraer datos con expresiones regulares requirió pruebas exhaustivas para manejar formatos inconsistentes. La librería xml.etree.ElementTree simplificó la escritura de autorizaciones.xml.
3. **Generación de gráficas**: Chart.js demandó procesar datos dinámicamente en JavaScript. Se implementó una capa intermedia en Django para formatear datos antes de enviarlos al frontend.
4. **Gestión del tiempo**: Los 17 días disponibles exigieron una planificación rigurosa. El plan

de trabajo en 4 fases (configuración, backend, frontend, documentación) aseguró un avance incremental.

Estos desafíos destacan la importancia de combinar herramientas modernas con una metodología estructurada. La experiencia refuerza la relevancia de POO para sistemas escalables y la necesidad de mejorar la formación en frameworks web.

Conclusiones

El sistema desarrollado cumple con los requisitos de la SAT, integrando POO, TDA, Flask, Django, y XML para procesar DTEs de manera eficiente. La arquitectura cliente-servidor, soportada por clases como ListaSolicitudes y ValidadorNIT, optimiza la modularidad y escalabilidad. Sin embargo, la curva de aprendizaje de Flask y Django representa un reto educativo que podría mitigarse con más recursos introductorios. Este proyecto evidencia el potencial de Python en aplicaciones tributarias y la importancia de la digitalización en Guatemala. Futuras investigaciones podrían explorar la integración con bases de datos relacionales o la automatización de pruebas unitarias. Se recomienda a estudiantes planificar cuidadosamente y usar foros colaborativos para superar obstáculos técnicos. ¿Cómo evolucionarán los sistemas tributarios con la adopción de tecnologías emergentes? Este trabajo invita a reflexionar sobre esa pregunta.

Referencias bibliográficas

- Armendáriz, D. (2020). *Flask Web Development*. O'Reilly Media.
- Date, C. J. (1991). *An Introduction to Database Systems*. Addison-Wesley.
- Django Software Foundation. (2024). *Django Documentation*. https://docs.djangoproject.com/
- Python Software Foundation. (2024). *Python 3.11 Documentation*. https://docs.python.org/3/
- W3Schools. (2024). *Chart.js Tutorial*. https://www.w3schools.com/js/js_chartjs.asp